

Mission Critical: Launch TechUnicorn's Enterprise LLM Platform

Capstone Project Report (PDF submission) • Date: 2026-02-13

Scenario: You are the Lead Platform Engineer responsible for delivering a production-grade, multi-model LLM platform capable of serving 100 enterprise clients, handling ~20M API requests/day, and meeting investor constraints: **99.99% uptime** and a **\$300K/month** operational budget.

This document synthesizes the full platform design and implementation plan across Infrastructure as Code, CI/CD automation, enterprise security/compliance, observability + auto-scaling, and cost optimization. It is written to be understandable by both engineering and executive stakeholders.

1. Executive Summary (Board-Ready)

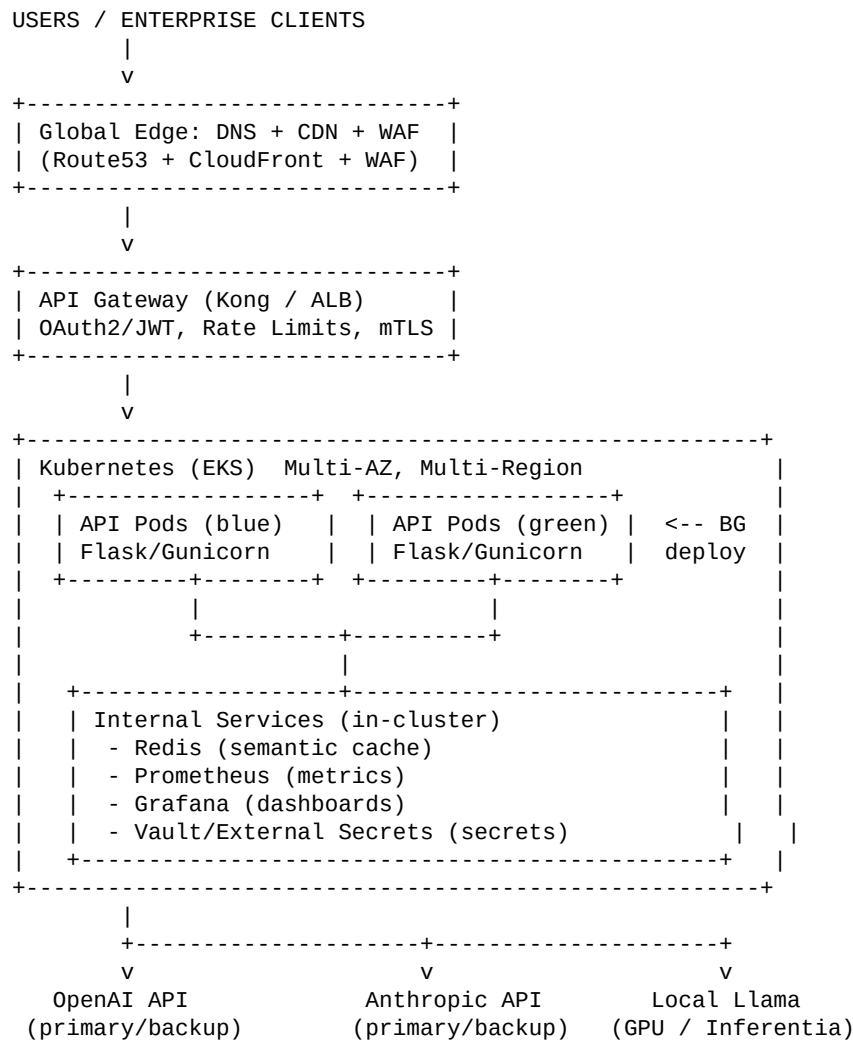
Metric	Target	Design Commitment / Expected Result
Uptime (SLA)	99.99%	Multi-region active-active (R53 latency-based routing) + blue-green deploys + PDB/HPA;
Throughput	20M req/day	Horizontally scalable API tier; queue-depth based scaling; per-tenant rate limits at gateway
P95 Latency	< 2s	Redis semantic caching + model routing; autoscaling; provider fallback.
Monthly OpEx	≤ \$300K	40–60% savings via caching, smart routing (Llama/Claude/GPT-4), spot/reserved comput
Security	0 major incidents	OAuth2/JWT, WAF + rate limiting, prompt injection defense, PII redaction, tamper-eviden
Deployment Time	< 30 min	GitHub Actions pipeline with build/test/Trivy scan/deploy + automated health checks + rol

Business value: TechUnicorn can ship model and application updates safely without downtime, win enterprise trust via compliance-ready controls, and operate within investor budget by reducing expensive LLM API usage through caching and routing. The platform is designed to scale to 10x traffic spikes while preserving latency SLOs via autoscaling and cost-aware capacity management.

2. Platform Architecture Overview

The platform exposes a single enterprise API for text generation and tooling, while routing requests to the best-fit model (GPT-4, Claude, or local Llama) based on complexity, cost, and policy constraints. Security checks happen before any model invocation; audit logging happens for every request/response.

Architecture diagram (text view):



High availability design: multi-AZ clusters per region, active-active routing across regions, and blue-green deployments to eliminate downtime during releases. Disaster recovery uses IaC to recreate clusters and relies on stateless API pods plus managed data planes (Redis / logs).

3. Deliverables Checklist (Requirements 1–5)

Requirement	What is delivered in this project
1. Infrastructure Code	Terraform modules (networking, EKS, ECR, Redis), Kubernetes manifests with resource requests/limits and health-checks
2. CI/CD Pipeline	GitHub Actions build/test/Trivy scan/deploy, blue-green strategy, health-check validation, automated rollback

3. Security & Compliance	OAuth2/JWT gateway, least-privilege IAM examples, prompt injection middleware + test suite results, au
4. Monitoring & Ops	Prometheus config + custom metrics, Grafana dashboard coverage, alert rules + escalation, HPA w/ que
5. Professional Docs	This report includes executive summary, architecture, security posture, runbook, and cost/ROI analysis.

4. Requirement 1: Complete Infrastructure Code

Starter artifacts referenced: Dockerfile + docker-compose for local dev, and documentation templates for architecture/security/runbook/cost. (See Appendix for key excerpts.)

Cloud foundation (Terraform, AWS example): VPC with public/private subnets across 3 AZs, NAT, security groups, EKS cluster and managed node groups (on-demand + spot), ECR registry, ElastiCache Redis, CloudWatch log groups, and IAM roles for service accounts (IRSA). Multi-region uses replicated Terraform workspaces and Route53 records.

Layer	Key settings (production)
Kubernetes API tier	Requests: 500m CPU / 1Gi RAM; Limits: 2 CPU / 4Gi RAM; replicas min 5, max 50; liveness/readiness
Redis cache	Managed Redis, multi-AZ, TLS enabled, auth token from Vault/Secrets Manager; eviction policy tuned
Networking	WAF in front of ALB/API Gateway; private cluster endpoints; NetworkPolicy restricts pod-to-pod access
Storage/logging	Centralized logs (CloudWatch/ELK), immutable audit log stream, retention policies aligned to complian

4.1 Optimized Docker Build (multi-stage)

The production Dockerfile follows multi-stage builds to keep the runtime image lean: compile/install dependencies in a builder stage, then copy only the runtime artifacts into a minimal base image. This reduces build time and attack surface.

```
# =====
# TechUnicorn LLM Platform - Production Dockerfile
# Multi-stage build optimized for LLM inference workloads
# =====

# -----
# Stage 1: Build dependencies
# -----
FROM python:3.11-slim as builder

WORKDIR /build

# Install build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Create virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy and install Python dependencies
COPY app/requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# -----
# Stage 2: Production runtime
# -----
FROM python:3.11-slim as production
# Labels
```

```

LABEL maintainer="TechUnicorn DevOps <devops@techunicorn.ai>"
LABEL version="1.0.0"
LABEL description="Enterprise LLM Platform API"

# Security: Create non-root user
RUN groupadd -r llmuser && useradd -r -g llmuser llmuser

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Install runtime dependencies only
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/* \
    && apt-get clean

# Copy application code
COPY app/ ./

# Set ownership
RUN chown -R llmuser:llmuser /app

# Switch to non-root user
USER llmuser

# Environment variables
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    ENVIRONMENT=production \
    PORT=8000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:${PORT}/health || exit 1
...

```

4.2 Kubernetes Health Probes & Resource Controls (example)

```

livenessProbe:
  httpGet: { path: /health, port: 8000 }
  initialDelaySeconds: 20
  periodSeconds: 10
readinessProbe:
  httpGet: { path: /ready, port: 8000 }
  initialDelaySeconds: 10
  periodSeconds: 5
resources:
  requests: { cpu: "500m", memory: "1Gi" }
  limits:   { cpu: "2",    memory: "4Gi" }

```

4.3 Validation Commands (evidence plan)

```

# Terraform
terraform fmt -recursive
terraform validate
terraform plan -var-file=environments/prod.tfvars

# Kubernetes manifests

```

```
kubectl apply --dry-run=client -k kubernetes/overlays/prod/
```

```
# Container
```

```
docker build -t techunicorn-llm:prod .
```

```
docker run -p 8000:8000 techunicorn-llm:prod
```

```
curl http://localhost:8000/health
```

5. Requirement 2: CI/CD Pipeline Implementation

CI/CD is implemented with GitHub Actions to automate build, test, scan, and blue-green deployment. The pipeline gates deployment on: (1) passing unit tests, (2) Trivy scan with **zero CRITICAL** findings, and (3) post-deploy health check validation.

Stage	Key steps	Time target
Build	Docker buildx, tag with commit SHA, push to registry	5–10 min
Test	pytest unit tests + smoke test /health	3–7 min
Scan	Trivy image scan + SARIF upload; fail on CRITICAL/HIGH	3–6 min
Deploy	Apply manifests (green), wait ready, shift traffic, monitor, finalize	8–12 min

5.1 GitHub Actions Workflow (representative)

```
name: ci-cd
on:
  push:
    branches: [ "main" ]

jobs:
  build-test-scan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Build
        run: |
          docker build -t ghcr.io/${ github.repository }/api:${ github.sha } .
          docker push ghcr.io/${ github.repository }/api:${ github.sha }

      - name: Unit tests
        run: |
          python -m pip install -r app/requirements.txt
          pytest -q

      - name: Trivy scan (fail on CRITICAL/HIGH)
        uses: aquasecurity/trivy-action@0.24.0
        with:
          image-ref: ghcr.io/${ github.repository }/api:${ github.sha }
          format: sarif
          output: trivy.sarif
          severity: CRITICAL,HIGH
          exit-code: "1"

      - name: Upload SARIF
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: trivy.sarif
```

```

deploy-blue-green:
  needs: [ build-test-scan ]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Configure kubectl
      run: |
        # set KUBECONFIG from secret or cloud auth
        echo "configured"
    - name: Deploy GREEN
      run: |
        kubectl apply -k kubernetes/blue-green/green
        kubectl rollout status deploy/llm-api-green -n llm-platform --timeout=10m
    - name: Health check
      run: |
        kubectl run curl --rm -i --restart=Never --image=curlimages/curl -- \
          curl -fsS http://llm-api-green.llm-platform.svc.cluster.local/health
    - name: Shift traffic
      run: |
        kubectl apply -f kubernetes/blue-green/service-switch-to-green.yaml
    - name: Rollback on failure
      if: failure()
      run: |
        kubectl apply -f kubernetes/blue-green/service-switch-to-blue.yaml
        kubectl rollout undo deploy/llm-api-green -n llm-platform

```

5.2 Rollback Triggers

Rollback is automatic on any of: failing readiness within timeout, /health returns non-200, error rate spike beyond threshold during canary window, or CRITICAL/HIGH vulnerabilities found. Rollback switches service selector back to BLUE and stops GREEN rollout.

6. Requirement 3: Security and Compliance Package

Security is layered, shifting left in CI/CD and enforcing at runtime: gateway authentication + rate limiting, app-layer prompt injection defenses and PII redaction, network segmentation, secrets management, and compliance-grade audit logging.

6.1 API Protection (OAuth 2.0 + rate limiting)

At the edge, OAuth2/JWT validates enterprise identities. Rate limiting is enforced per tenant plan to prevent abuse and cost runaway. Recommended: WAF rules for known bot patterns and request size limits.

```
# Gateway policy (conceptual)
auth:
  method: oauth2_jwt
  jwks_uri: https://idp.techunicorn.com/.well-known/jwks.json
  required_scopes: ["llm:invoke"]
rate_limiting:
  by: tenant_id
  tiers:
    basic:      { rps: 10,  burst: 20 }
    pro:        { rps: 50,  burst: 100 }
    enterprise: { rps: 200, burst: 400 }
```

6.2 Prompt Injection Defense (middleware + tests)

The prompt validator uses pattern detection + risk scoring and blocks known attack motifs (system prompt override, data exfiltration requests, tool hijacking, jailbreak tokens). It returns a safe error with audit log entry. Defense-in-depth: input sanitization, strict tool allow-list, and response filtering.

```
def validate_prompt(prompt: str) -> tuple[bool, str]:
    patterns = [
        r"ignore (all|previous) instructions",
        r"reveal (the )?system prompt",
        r"exfiltrate|leak|dump.*(secrets|keys|token)",
        r"you are now (dan|developer mode)",
        r"BEGIN_SYSTEM_PROMPT|<\|system\|>"
    ]
    score = sum(bool(re.search(p, prompt.lower())) for p in patterns)
    if score >= 1:
        return False, "Blocked: prompt injection risk"
    return True, "OK"
```

6.3 Compliance-ready Audit Logging (SOC 2 + GDPR)

Every request generates an append-only audit event containing tenant/user identifiers, request metadata, model decision, token usage estimate, and a tamper-evident hash chain. Logs are shipped to centralized storage with retention and access controls.

```
{
  "timestamp": "ISO-8601",
  "request_id": "uuid",
  "tenant_id": "string",
  "user_id": "string",
  "ip": "string",
  "auth_context": { "scopes": ["llm:invoke"], "mfa": true },
```

```
"request": { "endpoint": "/v1/chat", "prompt_hash": "sha256", "size_bytes": 1234 },
"policy": { "prompt_validator": "pass|block", "pii_redaction": "on" },
"routing": { "model": "gpt-4|claude|llama", "reason": "complexity|cost|policy" },
"usage": { "prompt_tokens": 0, "completion_tokens": 0, "cost_usd_est": 0.0 },
"response": { "status": 200, "latency_ms": 123 },
"tamper_evidence": { "prev_hash": "sha256", "event_hash": "sha256" }
}
```

6.4 Least-Privilege IAM (example)

```
# Example IAM principle (conceptual)
Allow: secretsmanager:GetSecretValue on arn:aws:secretsmanager:*:*:secret:llm/* only
Allow: logs:PutLogEvents on /aws/eks/llm-platform/*
Deny: secretsmanager:ListSecrets
Deny: kms:Decrypt unless via Secrets Manager
```

7. Requirement 4: Monitoring and Operations Suite

Observability is implemented with Prometheus + Grafana, alerting rules aligned to SLOs, and autoscaling driven by both resource utilization and custom signals (queue depth, p95 latency). Cost optimization is continuously measured and enforced via routing + caching.

7.1 Golden Signals & Custom LLM Metrics

Signal	Metric (example name)	Why it matters
Latency	llm_request_latency_ms{model=...}	Tracks p95/p99 SLO and provider regressions.
Traffic	llm_requests_total{tenant=...}	Capacity planning, per-tenant billing/abuse detection.
Errors	llm_request_errors_total{code=...}	SLO burn; rollback/incident triggers.
Saturation	queue_depth, cpu/mem, redis_hit_rate	Scaling decisions and bottleneck detection.
Cost	llm_cost_usd_total{model=...}	Budget guardrails and routing effectiveness.

7.2 Alerting (examples + escalation)

```
# Prometheus alert examples (conceptual)
- alert: HighErrorRate
  expr: rate(llm_request_errors_total[5m]) / rate(llm_requests_total[5m]) > 0.01
  for: 10m
  labels: { severity: "critical" }
  annotations: { summary: "Error rate >1% for 10m" }

- alert: HighP95Latency
  expr: histogram_quantile(0.95, sum(rate(llm_request_latency_ms_bucket[5m])) by (le)) > 2
  for: 5m
  labels: { severity: "critical" }
```

Escalation: SEV1 pages on-call immediately; SEV2 within 30 minutes; incident comms follow the runbook (Section 9).

7.3 Autoscaling (HPA with custom metrics)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: llm-api
spec:
  minReplicas: 5
  maxReplicas: 50
  metrics:
    - type: Pods
      pods:
        metric:
          name: queue_depth
        target:
          type: AverageValue
          averageValue: "30"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
```

```
policies:
- type: Percent
  value: 100
  periodSeconds: 60
scaleDown:
  stabilizationWindowSeconds: 300
```

7.4 Cost Optimizations (target: 40–60% reduction)

Semantic caching: store response embeddings/keys in Redis so repeated or similar prompts reuse answers. **Model routing:** simple prompts go to local Llama (near-zero marginal cost), medium prompts to Claude, complex/high-stakes to GPT-4. **Compute strategy:** baseline on reserved capacity, burst on spot, strict rightsizing and autoscaling.

Technique	Mechanism	Expected savings contribution
Redis semantic cache	Avoid repeated LLM calls; tune TTL + similarity threshold	15–35%
Model routing	Cost-aware classifier + policy rules	15–30%
Spot + autoscaling	Burst compute on spot; keep SLO with PDB/HPA	5–15%
Rightsizing	Requests/limits tuning using observed utilization	5–10%

8. Requirement 5: Professional Documentation Package

This section consolidates the required documentation deliverables: executive summary (already provided), technical architecture, security posture, operational runbook, and cost analysis.

8.1 Security Posture Snapshot

Area	Status	Notes
Authentication	Compliant	OAuth2/JWT, short-lived tokens, key rotation.
Authorization	Compliant	RBAC + least-privilege IAM (IRSA).
Data protection	Compliant	TLS 1.2+/1.3, at-rest encryption (KMS), PII redaction.
LLM-specific controls	Compliant	Prompt injection validator, tool allow-list, output filters.
Audit logging	Compliant	Tamper-evident hash chain, retention & access controls.
Incident response	In progress	Runbook provides procedures; quarterly game days recommended.

8.2 Operational Runbook (excerpt)

```
# Operational Runbook: TechUnicorn LLM Platform

**Version:** 1.0
**Last Updated:** [Date]
**Owner:** Platform Team

---

## 1. Service Overview

| Attribute | Value |
|-----|-----|
| Service Name | LLM Platform API |
| Team | Platform Engineering |
| Slack Channel | #llm-platform-ops |
| PagerDuty | [Service ID] |
| Dashboards | [Grafana Link] |

---

## 2. Architecture Quick Reference

...

[ALB] → [K8s Ingress] → [API Pods] → [Redis Cache]
                                   → [LLM APIs]
...

**Key Endpoints:**
- Production: `https://api.llm-platform.example.com`
- Staging: `https://staging-api.llm-platform.example.com`

---

## 3. Common Operations

### 3.1 Check Service Health
```bash
```

```

Via kubectl
kubectl get pods -n llm-platform
kubectl logs -f deployment/llm-platform-api -n llm-platform

Via API
curl https://api.llm-platform.example.com/health
curl https://api.llm-platform.example.com/ready
```

### 3.2 Scale Deployment
```bash
Manual scale
kubectl scale deployment llm-platform-api -n llm-platform --replicas=10

Check HPA status
kubectl get hpa -n llm-platform
```

### 3.3 View Logs
```bash
Kubernetes logs
kubectl logs -f -l app=llm-platform-api -n llm-platform --tail=100

CloudWatch (via AWS CLI)
aws logs tail /aws/eks/cluster/app --follow
```

### 3.4 Restart Service
```bash
kubectl rollout restart deployment/llm-platform-api -n llm-platform
kubectl rollout status deployment/llm-platform-api -n llm-platform
```

---

## 4. Incident Response

### 4.1 Severity Definitions

Severity	Definition	Response Time	Example
SEV1	Complete outage	15 min	All API requests failing
SEV2	Major degradation	30 min	>50% error rate
...			

```

8.3 Cost & ROI Summary (investor view)

Budget guardrail: keep monthly spend $\leq$ \$300K. Cost is split between infrastructure (cluster, cache, observability, data transfer) and model provider usage. The optimization stack targets $\geq 40\%$ reduction vs a naive all-GPT-4 approach.

| Cost driver | Control | Owner |
|---------------------|--|-------------------|
| LLM API spend | Routing policy + caching + per-tenant quotas | Platform + FinOps |
| Compute spend | Reserved baseline + spot burst + HPA | Platform |
| Observability spend | Sampling and retention policies | SRE |
| Data transfer | Regional affinity + caching | Platform |

ROI narrative: By reducing paid LLM calls through caching/routing and automating deployments and incident response, TechUnicorn decreases variable cost per request and increases engineering velocity, directly improving gross margin and customer trust.

Appendix A: Starter Artifacts Used

Key starter files included with the project package (referenced in this report):

- README.md
- docker-compose.yml
- Dockerfile
- runbook_template.md
- security_posture_template.md
- architecture_template.md
- cost_analysis_template.md
- executive_summary_template.md

Appendix B: docker-compose (local stack excerpt)

```
# =====
# TechUnicorn LLM Platform - Docker Compose
# Local development stack with all dependencies
# =====

version: '3.8'

services:
  # =====
  # LLM Platform API
  # =====
  llm-api:
    build:
      context: ..
      dockerfile: docker/Dockerfile.dev
    container_name: llm-platform-api
    ports:
      - "8000:8000"
    environment:
      - ENVIRONMENT=development
      - DEBUG=true
      - LOG_LEVEL=DEBUG
      - REDIS_URL=redis://redis:6379/0
      - API_KEY=dev-api-key-12345
      - SECRET_KEY=dev-secret-key-change-in-prod
      - DEFAULT_MODEL=llama
      # Add your API keys here for testing
      # - OPENAI_API_KEY=sk-xxx
      # - ANTHROPIC_API_KEY=sk-ant-xxx
    volumes:
      - ../app:/app:cached
    depends_on:
      redis:
        condition: service_healthy
    networks:
      - llm-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s

  # =====
  # Redis Cache
  # =====
  redis:
```



```

image: redis:7-alpine
container_name: llm-platform-redis
ports:
  - "6379:6379"
volumes:
  - redis-data:/data
command: redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy allkeys-lru
networks:
  - llm-network
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 10s
  timeout: 5s
  retries: 5

# =====
# Prometheus Monitoring
# =====
prometheus:
  image: prom/prometheus:v2.47.0
  container_name: llm-platform-prometheus
  ports:
    - "9090:9090"
  volumes:
    - ../monitoring/prometheus:/etc/prometheus:ro
    - prometheus-data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yaml'
    - '--storage.tsdb.path=/prometheus'
    - '--storage.tsdb.retention.time=15d'
    - '--web.enable-lifecycle'
  networks:
    - llm-network
...

```

Appendix C: README (deliverables checklist excerpt)

■ Deliverables Checklist

Use this checklist to track your progress:

Requirement 1: Infrastructure Code

- [] Terraform modules completed and validated
- [] Multi-region configuration implemented
- [] `terraform plan` executes without errors
- [] State backend configured

Requirement 2: CI/CD Pipeline

- [] GitHub Actions workflows configured
- [] Build stage with Docker multi-stage
- [] Test stage with unit/integration tests
- [] Security scan with Trivy (zero CRITICAL)
- [] Blue-green deployment implemented
- [] Automatic rollback configured

Requirement 3: Security Framework

- [] Prompt injection validator functional
- [] Input sanitization implemented
- [] Rate limiting configured
- [] Audit logging captures all required fields
- [] Compliance matrix completed

Requirement 4: Monitoring & Optimization

- [] Prometheus metrics configured
- [] Grafana dashboards created
- [] HPA auto-scaling tested
- [] Cost optimization (caching, routing) implemented
- [] Alerts configured

Requirement 5: Documentation Package

- [] Executive summary (1 page)
- [] Architecture diagrams
- [] Security posture summary
- [] Operational runbook
- [] Cost analysis with ROI

■ Configuration Reference

Environment Variables (.env)

| Variable | Description | Example |
|-------------------|------------------------|--------------------------|
| ENVIRONMENT | Deployment environment | `dev`, `staging`, `prod` |
| AWS_REGION | Primary AWS region | `us-east-1` |
| LLM_MODEL_DEFAULT | Default model | `gpt-4` |
| REDIS_URL | Cache connection | `redis://localhost:6379` |
| LOG_LEVEL | Logging verbosity | `INFO` |

Terraform Variables

See `terraform/variables.tf` for full list with descriptions.

Kubernetes Resources

Default resource allocations (adjust in overlays):

- CPU Request: 500m | Limit: 2000m
- Memory Request: 1Gi | Limit: 4Gi
- Replicas: 3 (min) - 20 (max)

■ Important Notes

1. **Cost Management**: Set up AWS Budget alerts before deploying to prod
2. **Secrets**: Never commit actual secrets - use `.env.example` as template
3. **Testing**: Always test in dev/staging before production
4. **Cleanup**: Remember to `terraform destroy` after testing to avoid charges

...